

What is Machine Learning?

Dr. Antonio Pellicani

Study Notes — Introduction to Machine Learning

1. Definition of Machine Learning

"Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed."

— Arthur Lee Samuel, AI pioneer, 1959

1.1 Traditional Programming vs Machine Learning

In traditional computer science, a programmer writes a program that encodes a set of explicit rules to solve a specific problem. The pipeline is:

Programmer + Rules + Data



Computer



Output

In Machine Learning, the machine infers the rules by learning from data. The pipeline is inverted:

Inputs + Desired Outputs



Computer



Program (Rules)

More formally: we want to implement an unknown function $f(x) \rightarrow y$, having access only to sample input-output pairs (training examples). Learning means incorporating information from these examples into the system.

1.2 Mitchell's Formal Definition

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."

— Tom M. Mitchell, Machine Learning, 1997

Applied to handwritten digit recognition:

Mitchell's Framework — Handwritten Digit Recognition	
Task T	Classifying handwritten digits from images
Performance P	Percentage of digits classified correctly
Experience E	Dataset of labeled digits (e.g., MNIST)

2. Why Use Machine Learning?

It is very hard to write programs that solve problems such as recognizing a handwritten digit. For instance:

- What distinguishes a '2' from a '7'?
- How does the human brain solve it?

Instead of writing a program by hand, we:

- Collect examples that specify the correct output for a given input.
- Feed them to a machine learning algorithm, which produces a program automatically.
- The resulting program may contain millions of parameters and generalizes to new, unseen inputs.

2.1 Application Domains

Domain	Task	Examples
Pattern recognition	Identify recurring structures	Speech, face recognition
Computer vision	Analyze image/video data	Detection, segmentation, depth estimation
Recommender systems	Personalized suggestions	Amazon, Netflix
Information retrieval	Find similar documents/images	Search engines
Robotics	Perception and planning	Autonomous robots
Anomaly detection	Identify unusual events	Credit card fraud, airport surveillance
Spam / fraud filtering	Adaptive classification	Email filters, fraud detection
Medical diagnosis	Classify patient data	Disease classification from imaging

3. Machine Learning Paradigms

3.1 Supervised Learning

The correct output is known for each training example. Goal: learn a mapping from inputs x to outputs y given a labeled training set $D = \{(x_i, y_i)\}$.

- Training set D has N examples.
- Each input x_i is a D -dimensional vector of features/attributes.
- Output y_i can be categorical → Classification problem.
- Output y_i can be real-valued → Regression problem.

Classification	Regression
Output: categorical (1-of-N)	Output: real-valued scalar
Examples: speech recognition	Examples: predicting market prices
Examples: object recognition	Examples: customer rating prediction
Examples: medical diagnosis	Examples: weather forecasting

3.2 Unsupervised Learning

Only input data $D = \{x_i\}$ is available. Goal: find interesting patterns, regularities, or structure in the data. No obvious error metric — a much less well-defined problem.

- **Association Analysis:** Discover strongly associated features/relationships (e.g., Market Basket Analysis — finding products frequently bought together).
- **Cluster Analysis:** Discover groups of closely related observations (e.g., grouping documents by topic: sport, economy...).
- **Anomaly Detection:** Identify observations with characteristics significantly different from the rest (outliers, deviations).
- **Dimensionality Reduction:** Compress data into a lower-dimensional representation while preserving structure (e.g., PCA).

3.3 Reinforcement Learning

No labeled examples. An agent interacts with an environment and receives a scalar reward signal.

- No labeled examples: feedback is only a scalar reward received after taking an action.
- Agent observes a state, takes an action, receives a reward, transitions to a new state.
- Goal: maximize cumulative reward over time.

Reinforcement Learning — Key Examples	
Game playing	Chess, Go (AlphaGo), video games
Robot locomotion	Teaching robots to walk/run
Self-driving cars	Navigation and decision-making

4. Types of ML Tasks — Formal Definitions

4.1 Classification

Goal: learn $f(x) \rightarrow y$ where $y \in \{1, \dots, C\}$, with C being the number of classes.

- Binary classification: $C = 2$
- Multiclass classification: $C > 2$
- Multi-label classification: class labels are not mutually exclusive

Formalized as function approximation: assume $y = f(x)$ for some unknown f . Learning estimates f from a labeled training set; predictions use $\hat{y} = \hat{f}(x)$.

Key goal: generalization — performing well on novel inputs not seen during training. Predicting training data is trivially easy.

Probabilistic Predictions

To handle ambiguous cases, we return a probability distribution over possible labels:

$$p(y \mid x, D) \quad \text{— vector of length } C$$

For binary classification it suffices to return $p(y = 1 \mid x, D)$, since $p(y=0) = 1 - p(y=1)$.

The best-guess label is:

$$\hat{y} = \operatorname{argmax}_c p(y = c \mid x, D) \quad \text{for } c = 0, \dots, C$$

4.2 Unsupervised Learning — Formal View

Formalized as density estimation: build a model $p(x_i \mid \theta)$ (unconditional), unlike supervised learning which is conditional density estimation $p(y_i \mid x_i, \theta)$.

Since x_i is a vector of features, we need multivariate probability models (more complex than supervised regression which predicts a single scalar y_i).

Discovering Clusters

Let K be the number of clusters.

- First goal: estimate $p(K | D) \approx K^* = \operatorname{argmax}_K p(K | D)$
- Second goal: assign each point i to its cluster: $z_i^* = \operatorname{argmax}_k p(z_i = k | x_i, D)$

5. ML vs Data Mining vs Statistics

Aspect	Data Mining	Statistics
Focus	Simple ML on very large DBs; historically limited by compute	Theoretical: proving estimation procedures are asymptotically unbiased
Output	Discover relationships in data (historically pejorative)	Elegant proof + formal guarantees
Relation to ML	Lines now blurred; many ML problems use huge datasets	ML is applying computational techniques to statistical problems
Current overlap	ML with 'AI flavor' (recognition, robotics) remains distinct	ML emphasizes demos producing impressive results over theory

6. Basic Concepts

6.1 Parametric vs Non-Parametric Models

Parametric Models	Non-Parametric Models
Fixed number of parameters	Parameters grow with training data
Faster to use at inference	More flexible
Stronger assumptions about data distribution	Often computationally intractable for large datasets
Example: Linear Regression	Example: K-Nearest Neighbors (KNN)

6.2 K-Nearest Neighbors (KNN)

KNN is a simple non-parametric classifier:

- Looks at the K points in the training set nearest to the test input x .
- Counts members of each class in this neighborhood.
- Returns the empirical fraction as the probability estimate.

$$p(y = c \mid x, D, K) = (1/K) \times \sum \mathbb{1}(y_i = c) \quad \text{for } i \in N_k(x, D)$$

where $N_k(x, D)$ are the K nearest points to x in D , and $\mathbb{1}(e)$ is the indicator function (1 if e is true, 0 otherwise).

6.3 Linear Regression

Linear Regression is the most widely used parametric model for regression. The response is a linear function of the inputs:

$$y(x) = w^T x + \varepsilon = \sum w_j x_j + \varepsilon \quad (j = 1, \dots, D)$$

where ε is the residual error between the linear prediction and the true response.

6.4 Overfitting

When fitting highly flexible models, we must be careful not to overfit: trying to model every minor variation in the input, which is more likely to be noise than true signal.

Scenario	Description	Consequence
Underfitting	Model too simple; does not capture the pattern	High training AND test error
Optimal fit	Balanced complexity; captures the true signal	Low training error, low test error
Overfitting	Model too complex; memorizes noise in training data	Low training error, HIGH test error

6.5 Model Selection

When multiple models solve the same problem with different complexity, we need a principled way to choose.

Why Not Use Training Error?

Training error can always be minimized by increasing model complexity. We care about generalization error: the expected misclassification rate on future data.

$$\text{err}(f, D) = (1/N) \times \sum \mathbb{1}(f(x_i) \neq y_i) \quad (i = 1, \dots, N)$$

Validation Set Approach

- Partition training data: ~80% training, ~20% validation.
- Fit all candidate models on the training set.
- Evaluate performance on the validation set and pick the best.
- Refit the best model on all available data.

Cross-Validation (CV)

A more robust solution to the model selection problem:

- Split training data into K folds.
- For each fold $k \in \{1, \dots, K\}$: train on all folds except the k -th, test on the k -th.
- Compute the average error over all folds as a proxy for test error.
- Special case: $K = N \rightarrow$ Leave-One-Out Cross-Validation (LOOCV).

Cross-Validation Summary	
Method	K-Fold Cross-Validation
Parameter K	Typically $K = 5$ or $K = 10$
Special case	$K = N \rightarrow$ Leave-One-Out CV (LOOCV)
Advantage	Uses all data for both training and validation
Use	Proxy for generalization error; used for model selection

6.6 The No Free Lunch Theorem

"All models are wrong, but some models are useful."
 — **George Box, 1987**

- There is no universally best model in machine learning.
- A set of assumptions that works well in one domain may work poorly in another.
- Much of ML research is about devising different models and algorithms for different problems.
- The no free lunch theorem formalizes this: without prior assumptions about the data, no learning algorithm is universally superior.

7. Summary Table — ML Paradigms at a Glance

Paradigm	Goal	Examples
Supervised Learning	Learn $f(x)$ to y from labeled data	Classification, Regression
Unsupervised Learning	Find structure/patterns in data	Clustering, Anomaly detection
Reinforcement Learning	Maximize cumulative reward	Game playing, Robotics

End of Lecture